File Handling

File handling in C enables us to create, update, read, and delete the files stored on the local file system through our C program. The following operations can be performed on a file.

- Creation of the new file
- Opening an existing file
- Reading from the file
- Writing to the file
- Deleting the file

Functions for file handling

There are many functions in the C library to open, read, write, search and close the file. A list of file functions are given below:

No.	Function	Description
1	fopen()	opens new or existing file
2	fprintf()	write data into the file
3	fscanf()	reads data from the file
4	fputc()	writes a character into the file
5	fgetc()	reads a character from file
6	fclose()	closes the file
7	fseek()	sets the file pointer to given position
8	fputw()	writes an integer to file
9	fgetw()	reads an integer from file
10	ftell()	returns current position
11	rewind()	sets the file pointer to the beginning of the file

---

We can use one of the following modes in the fopen() function.

Mode	Description
R	opens a text file in read mode
w	opens a text file in write mode
A	opens a text file in append mode
r+	opens a text file in read and write mode
w+	opens a text file in read and write mode

a+	opens a text file in read and write mode
rb	opens a binary file in read mode
wb	opens a binary file in write mode
ab	opens a binary file in append mode
rb+	opens a binary file in read and write mode
wb+	opens a binary file in read and write mode
ab+	opens a binary file in read and write mode

Example to input data in file

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int num;
   FILE *fptr;

   fptr = fopen("C:\\sss.txt","w");

   if(fptr == NULL)
   {
      printf("Error!");
      exit(1);
   }

   printf("Enter num: ");
   scanf("%d",&num);

   fprintf(fptr,"%d",num);
   fclose(fptr);

   return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int num;
   FILE *fptr;

   if ((fptr = fopen("C:\\sss.txt","r")) == NULL){
       printf("Error! opening file");

       exit(1);
   }

   fscanf(fptr,"%d", &num);

   printf("Value of n=%d", num);
   fclose(fptr);

   return 0;
}
```

Difference between Algorithm and Pseudocode

Algorithm	Pseudocode
An Algorithm is used to provide a solution to a particular problem in form of a well-defined step-based form.	A Pseudocode is a step-by-step description of an algorithm in code-like structure using plain English text.
An algorithm only uses simple English words	Pseudocode also uses reserved keywords like if-else, for, while, etc.
These are a sequence of steps of a solution to a problem	These are fake codes as the word pseudo means fake, using code like structure and plain English text
There are no rules to writing algorithms	There are certain rules for writing pseudocode
Algorithms can be considered pseudocode	Pseudocode cannot be considered an algorithm
It is difficult to understand and interpret	It is easy to understand and interpret

Difference between Flowchart and Pseudocode

Flowchart	Pseudocode
A Flowchart is pictorial representation of flow of an algorithm.	A Pseudocode is a step-by-step description of an algorithm in code like structure using plain English text.
A Flowchart uses standard symbols for input, output decisions and start stop statements. Only uses different shapes like box, circle and arrow.	Pseudocode uses reserved keywords like if-else, for, while, etc.
This is a way of visually representing data, these are nothing but the graphical representation of the algorithm for a better understanding of the code	These are fake codes as the word pseudo means fake, using code like structure but plain English text instead of programming language
Flowcharts are good for documentation	Pseudocode is better suited for the purpose of understanding

Call by value in C

- In call by value method, the value of the actual parameters is copied into the formal parameters. In other words, we can say that the value of the variable is used in the function call in the call by value method.
- In call by value method, we can not modify the value of the actual parameter by the formal parameter.
- In call by value, different memory is allocated for actual and formal parameters since the value of the actual parameter is copied into the formal parameter.
- The actual parameter is the argument which is used in the function call whereas formal parameter is the argument which is used in the function definition.

```
1. #include<stdio.h>
2. void change(int num) {
3.     printf("Before adding value inside function num=%d \n",num);
4.     num=num+100;
5.     printf("After adding value inside function num=%d \n", num);
6. }
7. int main() {
8.     int x=100;
9.     printf("Before function call x=%d \n", x);
10.    change(x);//passing value in function
11.    printf("After function call x=%d \n", x);
12. return 0;
13. }
```

Call by reference in C

- In call by reference, the address of the variable is passed into the function call as the actual parameter.
- The value of the actual parameters can be modified by changing the formal parameters since the address of the actual parameters is passed.
- In call by reference, the memory allocation is similar for both formal parameters and actual parameters. All the operations in the function are performed on the value stored at the address of the actual parameters, and the modified value gets stored at the same address.

```
• #include<stdio.h>
• void change(int *num) {
•     printf("Before adding value inside function num=%d \n",*num);
•     (*num) += 100;
•     printf("After adding value inside function num=%d \n", *num);
• }
• int main() {
•     int x=100;
•     printf("Before function call x=%d \n", x);
•     change(&x);//passing reference in function
•     printf("After function call x=%d \n", x);
• return 0;
• }
```

String Functions

1. strcat() Function

The strcat() function in C is used for string concatenation. It will append a copy of the source string to the end of the destination string.

```
int main()
{
    char dest[50] = "This is an";
    char src[50] = " example";
      strcat(dest, src);
      printf("dest After: %s", dest);

    return 0;
}Out put will be:- "This is an example"
```

2. strlen() Function

The strlen() function calculates the length of a given string. It doesn't count the null character '\0'

```
int main()
{

    char str[] = "CompuocmCollege";

    int length = strlen(str);


    printf("Length: %d\n", length);

    return 0;
} output:-15
```

3. strcmp() Function

The strcmp() is a built-in library function in C. This function takes two strings as arguments and compares these two strings lexicographically (Character By Charatcetr)

Return Value

- If str1 is less than str2, the return value is less than 0.
- If str1 is greater than str2, the return value is greater than 0.
- If str1 is equal to str2, the return value is 0.

```
int main()
{
    char str1[] = "Compucom";
    char str2[] = "Front";
    char str3[] = "Compucom";
      // Compare 'str1' and 'str2' using strcmp() function and
    // store the result in 'result1'
```

```
•       int result1 = strcmp(str1, str2);
•       // Compare 'str2' and 'str3' using strcmp() function and
•       // store the result in 'result2'
•       int result2 = strcmp(str2, str3);
•       // Compare 'str1' and 'str1' using strcmp() function and
•       // store the result in 'result3'
•       int result3 = strcmp(str1, str1);
•      printf("Comparison of str1 and str2: %d\n", result1);
•       printf("Comparison of str2 and str3: %d\n", result2);
•       printf("Comparison of str1 and str1: %d\n", result3);
•
•   }Out 1,-1,0
```

4. strcpy

The strcpy() is a standard library function in C and is used to copy one string to another. In C, it is present in **<string.h>** header file.

```
int main()
{

    char source[] = "Compucom";
    char dest[20];

      strcpy(dest, source);

    printf("Source: %s\n", source);
    printf("Destination: %s\n", dest);

    return 0;
}
```

.